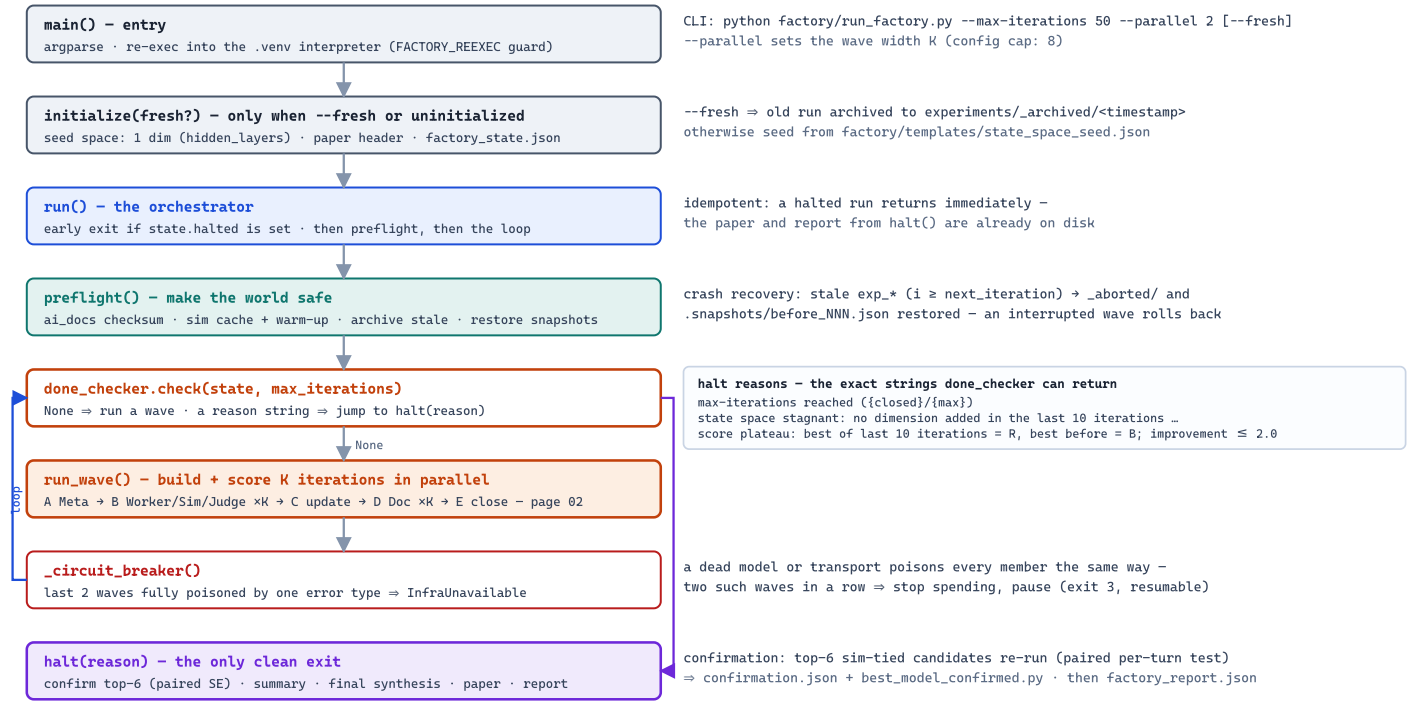


01 · THE RUN, END TO END

THE GOLDEN RULE – “The harness is deterministic; the work is creative.” Python owns the loop, every stop decision, every model choice and every file write. Agents are pure functions from a whitelisted prompt to JSON.

What actually happens when you type `python factory/run_factory.py`. Every box below is Python code in `run_factory.py`; LLM agents exist only inside `run_wave()` (page 02). There is exactly one clean way out — `halt(reason)` — and three ways to stop early.

RUN TIMELINE – `main()` → `exit` code



factory_state.json – one JSON file is the whole run

```
{
  "schema_version": 1, "created": "...",
  "next_iteration": 0,      # == iterations closed
  "iterations": [],        # one row per closed iteration (p.05)
  "scores": {},            # judge scores – plateau rule
  "sim_scores": {},        # deterministic sim scores
  "best": null,            # {iteration, score, ...}
  "last_judge_score": null, # router input R3
  "meta_invalid_streak": 0, # router input R4
  "last_dimension_added_iteration": -1, # stagnation rule
  "poisoned": [],          # exhausted iterations
  "canaries": {}, "halted": null,
  "ai_docs_checksum": "sha256...", # read-only guard
  "llm_usage": {"calls": 0, "cost_usd": 0.0}
  # runtime additions: waves · fingerprints · baselines
}
```

EXIT CODES – four ways the process ends

exit	meaning	state on disk
0	halt(reason) ran to completion	halted=<reason> · confirmation.json · factory_report.json
2	MacroFail – isolation / read-only violation	macro_fail.json · stale exp_* archived at next start
3	InfraUnavailable – LLM transport down	resumable – snapshots intact; re-run the same command
130	KeyboardInterrupt	resumable – same as exit 3

The loop never idles: every wave either closes K leaderboard rows or poisons them — both are recorded, both feed the next Meta call. There is no other path through the factory.

02 · ONE WAVE AS A SEQUENCE DIAGRAM

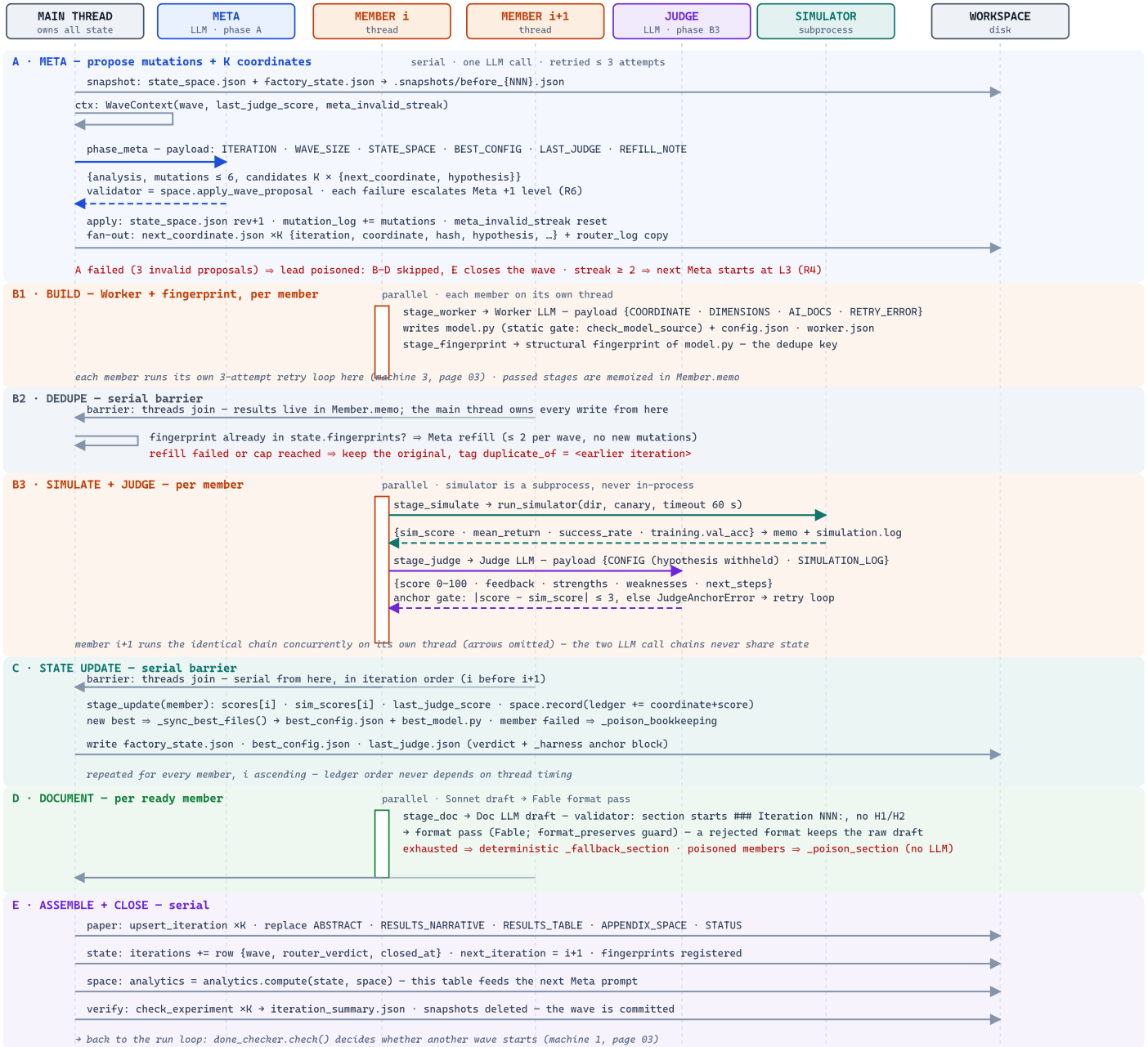
A wave builds **K** = --parallel iterations at once. Time flows down; bars on lifelines mark busy threads; gray/blue/teal/purple bands are **serial — main thread only**, orange/green bands are **parallel — member threads**. Threads only compute: every shared-state write happens on the main thread at a barrier, in iteration order, so the outcome never depends on which thread finishes first.

A Meta: 1 call, ≤3 attempts

B1 build ×K · B2 dedupe · B3 simulate+judge ×K

C ledger update · D document ×K · E paper + close

refills ≤ 2/wave · sim timeout 60 s · judge anchored ±3

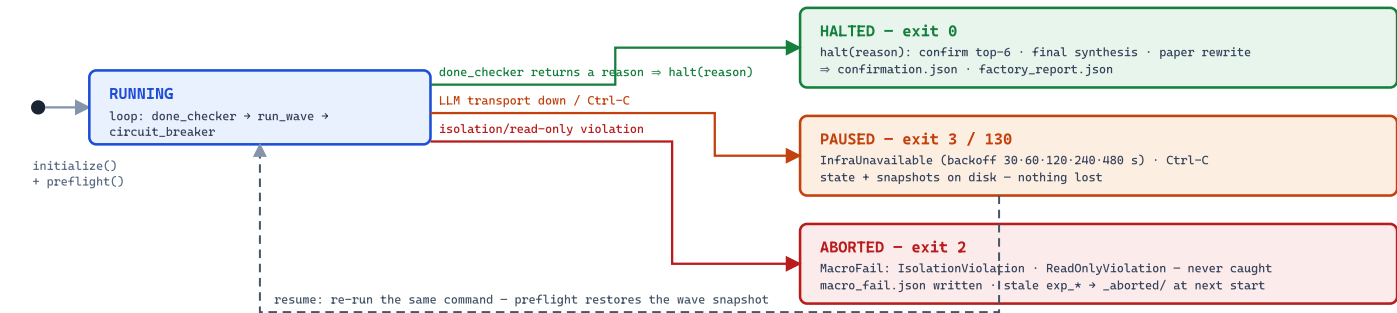


— call / file write · → dashed = return · white bar on a lifeline = thread busy · bands: blue/gray/teal/purple = serial (main thread) · orange/green = parallel (member threads)

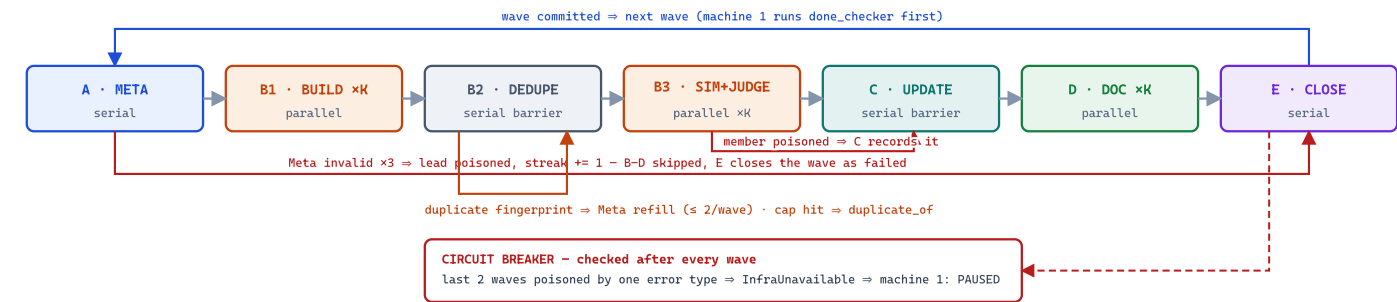
03 · THE THREE NESTED STATE MACHINES

The factory is three state machines stacked. A failure at a lower level is caught, retried and escalated there; it only travels upward when local retries exhaust. **Iteration** failures feed the wave machine; two identically-poisoned **waves** trip the circuit breaker and pause the **run**.

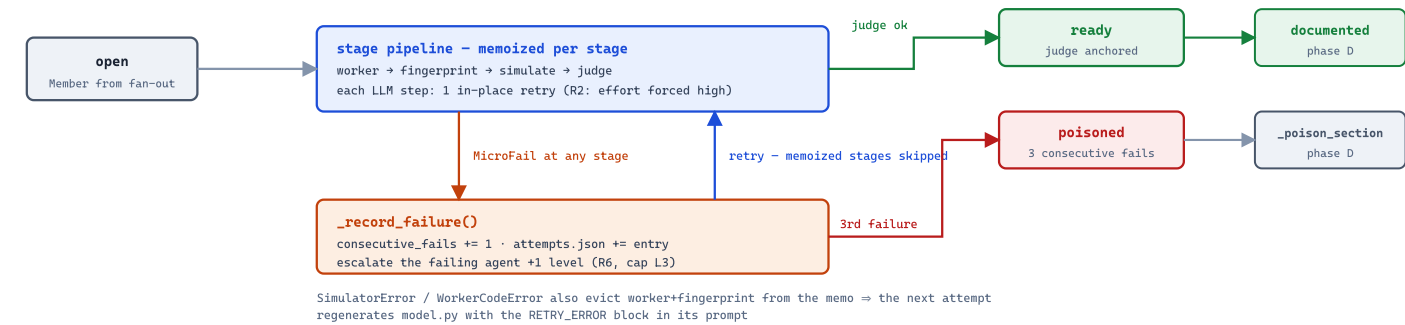
MACHINE 1 · THE RUN – process lifetime, main()



MACHINE 2 · THE WAVE – phases A-E, run_wave()



MACHINE 3 · THE ITERATION – one member thread, member_core()

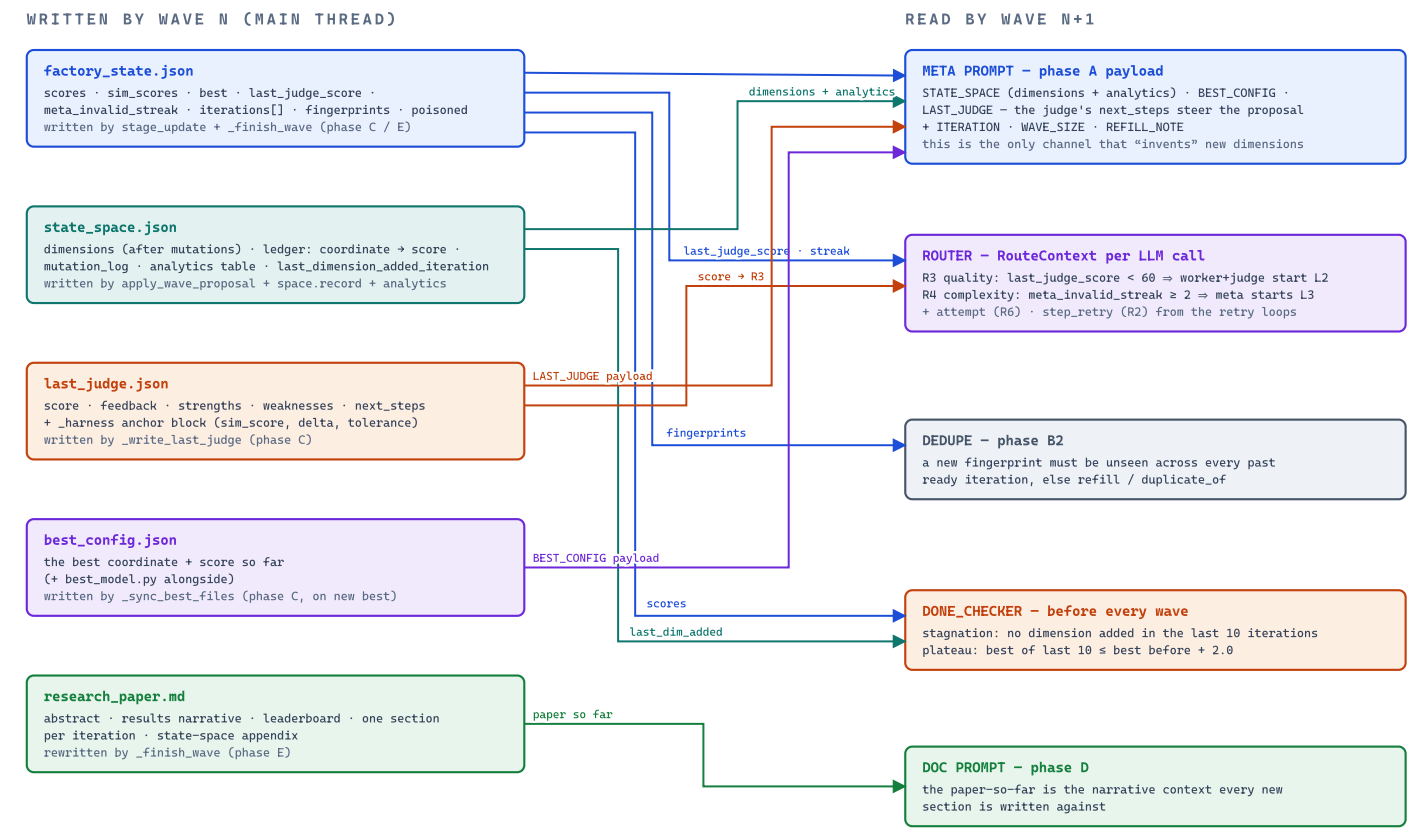


escape hatches: `OSError` ⇒ `InfraUnavailable` (machine 1: **PAUSED**) · `unexpected Exception` ⇒ `HarnessStageError`, treated as `MicroFail`
· `MacroFail` ⇒ machine 1: **ABORTED** – never caught inside an iteration

04 · THE FEEDBACK LOOP BETWEEN WAVES

Nothing resets between waves except the member threads. Wave N's outputs are written to disk in phases C/E and read back as wave N+1's inputs — the **Meta prompt**, the **router context**, the **dedupe set** and the **halt test**. This is the whole learning loop: there is no other memory.

WAVE N → WAVE N+1 WIRING – producer artifact → consumer



ROUTER RECAP – the only place a model is chosen · ladder L0-L3

rule	name	trigger → effect (applied in this order)
R1	baseline	every core agent starts at L0
R3	quality	last judge score < 60 ⇒ worker + judge start at L2 next iteration
R4	complexity	meta invalid streak ≥ 2 ⇒ meta starts at L3
R6	iteration	each iteration-level failure bumps the failing agent +1 level (cap L3) – per-iteration EscalationTracker, so parallel iterations never interact
R2	micro	in-place step retry ⇒ effort forced high on the same model
R5	fable	fable-5.1 only for (doc, format) – anything else raises RoutingViolationError

level	model	effort	used for
L0	sonnet-5	standard	default for every core agent
L1	sonnet-5	high	first escalation step / in-place retry
L2	opus-5.5	standard	quality floor after a low judge score (R3)
L3	opus-5.5	high	meta after 2 invalid waves (R4) · retry ceiling (R6)
–	fable-5.1	–	doc format pass only (R5)

05 · ONE ITERATION'S PAPER TRAIL

Every arrow on the previous pages is a file. Below: the real shapes for a healthy iteration (**exp_000**, scored) and a poisoned one (**exp_001**, exhausted its 3 attempts). **Keys are exact; values are illustrative.**

exp_000 · READY – fan-out → build → score

```
# written by _fan_out_meta (phase A)
experiments/exp_000/next_coordinate.json
{
  "iteration": 0, "experiment": "exp_000",
  "coordinate": {"hidden_layers": [64, 64]},
  "coordinate_hash": "9f2c1ab34d55",
  "hypothesis": "Two 64-unit layers give the
    policy capacity to separate the 6 actions.",
  "wave_iterations": [0, 1], "wave_position": 0,
  "mutations_applied": [], "dimensions_added": []
}
```

```
# written by stage_worker (phase B1)
experiments/exp_000/config.json
{
  "experiment": "exp_000", "iteration": 0,
  "canary": "exp_000::7f3a9c...",
  "coordinate": {"hidden_layers": [64, 64]},
  "coordinate_hash": "9f2c1ab34d55",
  "dimension_definitions": {"hidden_layers": {
    "type": "categorical", "values": [[64, 64]],
    "description": "Widths of the MLP hidden..."},
  "hypothesis": "Two 64-unit layers...",
  "worker": {"design_notes": "...",
    "model_py_sha256": "sha256...",
    "static_gate": {"lines": 41, ...}}
}
```

```
# written by stage_judge (phase B3)
experiments/exp_000/judge.json
{
  "score": 62,
  "feedback": "...20-2500 chars...",
  "strengths": [...], "weaknesses": [...],
  "next_steps": [...], # next Meta reads these
  "_harness": { # added by the harness
    "experiment": "exp_000", "iteration": 0,
    "sim_score": 61.3,
    "anchor_delta": 0.7, "anchor_tolerance": 3 }
}
```

exp_001 · POISONED – 3 attempts exhausted

```
# appended by _record_failure (phases B1-B3)
experiments/exp_001/attempts.json
[
  {"attempt": 0, "stage": "worker", "agent": "worker",
    "error_type": "WorkerCodeError",
    "error": "static gate: forbidden symbol..."},
  {"attempt": 1, "stage": "simulate", "agent": "worker",
    "error_type": "SimulatorError",
    "error": "model.py raised: shape (6 vs 5)..."},
  {"attempt": 2, "stage": "simulate", "agent": "worker",
    "error_type": "SimulatorTimeout",
    "error": "exceeded the 60 s budget"}
]
# 3 consecutive fails ⇒ status = poisoned
```

```
# written by _finish_wave (phase E)
experiments/exp_001/iteration_summary.json
{
  "iteration": 1, "experiment": "exp_001",
  "status": "poisoned",
  "score": null, "sim_score": null, "mean_return": null,
  "coordinate_hash": "77aa09bc21ef", "dims_added": 0,
  "attempts": 3, "duplicate_of": null, "refills": [],
  "failures": [{"attempt": 0, "stage": "worker", ...}, ...],
  "wave": 0, "router_verdict": "...",
  "closed_at": "2026-10-02T14:31:07",
  "micro_done": {"micro_done": false, ...} # verify_run
}
```

```
# factory_state.json after wave 0 (fragment)
{
  "next_iteration": 2,
  "iterations": [ {row exp_000...}, {row exp_001...} ],
  "scores": {"0": 62}, "sim_scores": {"0": 61.3},
  "best": {"iteration": 0, "score": 62, ...},
  "last_judge_score": 62, "poisoned": [1],
  "waves": 1
}
# ⇒ wave 1's Meta prompt carries exactly this state
```

WHO WRITES WHAT – file → the only function allowed to write it

file	writer	phase · note
exp_*/next_coordinate.json	_fan_out_meta	A · one per member; refills write next_coordinate.rejected_N.json alongside
exp_*/model.py · config.json · worker.json	stage_worker	B1 · model.py carries a header: coordinate hash + “generated by the Worker Agent”
exp_*/simulation.log · simulation_summary	simulator subprocess	B3 · written by the subprocess itself – the factory never trains in-process
exp_*/judge.json	stage_judge	B3 · verdict + _harness anchor block (sim_score, delta, tolerance)
exp_*/attempts.json	_record_failure	B1-B3 · appended on every MicroFail; 3 entries ⇒ poisoned
exp_*/iteration_summary.json	_finish_wave + verify_run	E · leaderboard row + micro_done checks – the per-iteration audit
workspace/factory_state.json	save_state	C/E · written on every mutation; the whole run is recoverable from it
workspace/research_paper.md	_finish_wave / halt	E · marker blocks replaced wholesale; sections upserted per iteration

A poisoned iteration is not wasted work: it yields a leaderboard row, a deterministic paper section (**_poison_section**), and a ledger entry the next Meta call can see. Failure is data.